

However, I strongly believe that directly accessing an instance variable of an object is an extremely dangerous practice, as it breaks the core idea of encapsulation. To illustrate why this is the case, let's take a look at a few example scenarios where direct access can be problematic.

Changing the Name of an Instance Variable

The first problem with direct access is that changing the name of an instance variable will break any client code that uses the original name directly. This can happen when the developer of a class decides that the initial choice of the name of the variable was not optimal, for reasons such as the following:

- The name doesn't describe the data it represents clearly enough.
- The variable is a Boolean, and they want to swap what `True` and `False` represent by renaming the variable (for example, `closed` to `open`, `allowed` to `disallowed`, `active` to `disabled`).
- There was a spelling or capitalization mistake in the original name.
- The variable was originally a Boolean, but they later realize that they need to represent more than two values.

In any of these cases, if the developer changes the name of an instance variable in the class from `self.<originalName>` to `self.<newName>`, then any client software that uses the original name directly will break.

Changing an Instance Variable into a Calculation

A second situation where direct access is problematic is when the code of a class needs to change to meet new requirements. Suppose that when writing a class, you use an instance variable to represent a piece of data, but the functionality changes so that you need an algorithm to compute a value instead. Take our `Account` class from Chapter 4, for example. To make our bank accounts more realistic, we might want to add an interest rate. You might think this is a simple matter of adding an instance variable for the interest rate named `self.interestRate`. Then, using the direct access approach, client software could access this value of an `Account` object using: